

CERTIFICAÇÃO LINUX

NGINX e Apache

Configuração contra XSS e SQL Injection — headers,
WAF e boas práticas

Prof. Uirá Ribeiro

Material de apoio do curso Certificação Linux.

Todos os direitos reservados.

Manual de Configuração — NGINX e Apache contra XSS e SQL Injection

Escopo: hardening na camada de **servidor web / proxy reverso** (headers de segurança, WAF, filtragem de requisições) para reduzir a superfície e o impacto de XSS e SQL Injection. Isso é **defesa em profundidade** — não substitui a correção na aplicação (queries parametrizadas, output encoding contextual). Ver seção 4.

Sumário

1. Fundamentos
 2. NGINX
 - 2.7 — Bloco de hardening legado (estilo WordPress), anotado linha a linha
 3. Apache HTTP Server
 4. O que a camada de servidor web NÃO resolve
 5. Cheat-sheet de diretivas
 6. Referências
-

1. Fundamentos

1.1 — XSS (Cross-Site Scripting) em uma frase

O atacante consegue injetar HTML/JavaScript que é refletido (**Reflected**), armazenado (**Stored**) ou processado no DOM (**DOM-based**) e executado no navegador de outra vítima, roubando sessão, credenciais ou executando ações em nome dela.

1.2 — SQL Injection em uma frase

O atacante consegue inserir fragmentos de SQL em um parâmetro que é concatenado diretamente numa query, alterando a lógica da consulta — de vaziar dados de outros usuários até derrubar tabelas ou obter execução de comandos, dependendo do banco e dos privilégios.

1.3 — Onde o servidor web entra nisso

O NGINX e o Apache não sabem nada sobre a lógica da sua aplicação — eles não conseguem "corrigir" uma query vulnerável ou uma página que ecoa input sem escapar. O que eles **podem** fazer é adicionar camadas de defesa **antes** da requisição chegar à aplicação, ou **depois** que a resposta é gerada:

CAMADA	O QUE FAZ	ONDE FICA NESSE MANUAL
Headers de segurança	Reduz o <i>impacto</i> de um XSS que passou (CSP, X-Frame-Options, etc.)	2.1 / 3.1
WAF (ModSecurity + OWASP CRS, ou NAXSI)	Inspeciona o corpo/query da requisição e bloqueia padrões conhecidos de ataque	2.2 / 2.3 / 3.2
Filtros diretos (regex em <code>map</code> / <code>RewriteCond</code>)	Bloqueio rápido e grosseiro de assinaturas óbvias — último recurso	2.4 / 3.3
Rate limiting / limites de tamanho	Dificulta automação de scanners e fuzzing de payloads	2.5

A ordem de prioridade real de eficácia é: **WAF com regras mantidas (CRS) > headers de segurança > regex artesanal**. Regex isolada é a mais fácil de contornar (encoding, comentários SQL, maiúsculas/minúsculas, espaços alternativos) — trate-a como um curativo, não como a defesa.

2. NGINX

2.1 — Headers de segurança

Adicione em `http {}`, `server {}` ou `location {}` (mais específico vence). Sempre use `always` para que o header seja enviado mesmo em respostas de erro (401/403/500) — sem `always`, o NGINX omite o header nessas respostas.

```
# /etc/nginx/conf.d/security-headers.conf

add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Permissions-Policy "geolocation=(), microphone=(), camera=()" always;

# CSP é a defesa mais eficaz contra o IMPACTO de um XSS refletido/armazenado:
# mesmo que um <script> malicioso seja injetado, o navegador recusa executá-lo
# se a origem do script não estiver na allowlist abaixo.
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self'
'unsafe-inline'; object-src 'none'; base-uri 'self'; frame-ancestors 'none';" always;
```

O que cada diretiva faz:

- **X-Content-Type-Options: nosniff** — impede que o navegador "adivinhe" o tipo de um arquivo (MIME sniffing). Sem isso, um upload de imagem malformado pode ser interpretado como HTML/JS e executado.
- **X-Frame-Options: DENY** — impede que a página seja carregada dentro de um `<iframe>` de outro site (mitiga clickjacking, frequentemente combinado com XSS para roubo de clique).
- **Content-Security-Policy (CSP)** — a diretiva mais importante contra XSS. `script-src 'self'` faz o navegador **recusar executar** qualquer `<script>` que não venha do próprio domínio, incluindo scripts injetados via XSS refletido. Evite `'unsafe-inline'` em `script-src` sempre que possível — ele reabre a porta que a CSP existe para fechar.
- **Referrer-Policy** — evita vaziar a URL completa (que pode conter tokens em query string) para sites de terceiros via header `Referer`.
- **Não** inclua mais o header `X-XSS-Protection`: o filtro que ele ativava foi removido do Chrome/Edge/Safari por gerar vulnerabilidades próprias; se precisar declará-lo por compatibilidade legada, use `X-XSS-Protection: 0` (desativado explicitamente), nunca `1; mode=block`.

2.2 — ModSecurity + OWASP Core Rule Set (CRS)

O ModSecurity é o WAF de código aberto de referência. Com o **OWASP CRS**, ele já vem com centenas de regras testadas contra XSS (arquivo `REQUEST-941-*`) e SQL Injection (`REQUEST-942-*`) — muito mais robustas do que qualquer regex que se escreva à mão.

Instalação (Debian/Ubuntu, NGINX ≥ 1.19 com módulo dinâmico):

```
sudo apt update
sudo apt install libmodsecurity3 libmodsecurity-dev nginx-plugin-modsecurity
```

Se o pacote pronto não existir na sua distro, compile o conector `ModSecurity-nginx` contra o mesmo binário do NGINX instalado (processo documentado no repositório oficial `owasp-modsecurity/ModSecurity-nginx`).

Carregar o módulo (topo de `/etc/nginx/nginx.conf`):

```
load_module modules/nginx_http_modsecurity_module.so;
```

Ativar dentro do bloco `http` ou `server`:

```
http {  
    modsecurity on;  
    modsecurity_rules_file /etc/nginx/modsec/main.conf;  
    ...  
}
```

`/etc/nginx/modsec/main.conf` — orquestra a configuração base + o CRS:

```
Include /etc/nginx/modsec/modsecurity.conf  
Include /etc/nginx/modsec/crs-setup.conf  
Include /etc/nginx/modsec/coreruleset/rules/*.conf
```

`/etc/nginx/modsec/modsecurity.conf` — parta do arquivo `recommended` oficial e ajuste o essencial:

```
SecRuleEngine On  
SecRequestBodyAccess On  
SecResponseBodyAccess Off  
SecRequestBodyLimit 13107200  
SecAuditEngine RelevantOnly  
SecAuditLog /var/log/nginx/modsec_audit.log  
SecAuditLogParts ABIJDEFHZ
```

- `SecRuleEngine On` — bloqueia de fato (`DetectionOnly` só registra em log, útil na primeira semana de rollout para calibrar falsos positivos antes de bloquear em produção).
- `SecRequestBodyAccess On` — necessário para inspecionar corpo de `POST` (onde a maioria dos payloads de SQLi em formulários trafega).

Baixar o CRS:

```
sudo git clone https://github.com/coreruleset/coreruleset.git /etc/nginx/modsec/coreruleset  
sudo cp /etc/nginx/modsec/coreruleset/crs-setup.conf.example /etc/nginx/modsec/crs-setup.conf
```

Regra própria de exemplo (para entender a sintaxe — na prática, o CRS já cobre isso com muito mais nuance):

```
# Bloqueia tentativas óbvias de XSS via parâmetro
SecRule ARGS "@detectXSS" \
    "id:1001,phase:2,deny,status:403,log,msg:'Possible XSS Attack Detected'"

# Bloqueia tentativas óbvias de SQL Injection via parâmetro
SecRule ARGS "@detectSQLi" \
    "id:1002,phase:2,deny,status:403,log,msg:'Possible SQL Injection Detected'"
```

`@detectXSS` e `@detectSQLi` são operadores baseados em `libinjection` — a mesma biblioteca usada pelo CRS internamente. Eles analisam a **estrutura** do input (não apenas por substring), o que reduz falsos positivos comparado a um `@rx <script>`.

2.3 — Alternativa nativa: NAXSI

O **NAXSI** ("Nginx Anti XSS & SQL Injection") é um WAF que roda como módulo nativo do NGINX, sem dependência do ModSecurity. Ele funciona por **whitelisting**: aprende quais requisições legítimas disparam "scores" de regras genéricas e você cria exceções (`BasicRule wl:<id>`) para elas — inverso do modelo de blacklist do ModSecurity.

```
http {
    include /etc/nginx/naxsi_core.rules;

    server {
        location / {
            SecRulesEnabled;
            DeniedUrl "/RequestDenied";

            CheckRule "$SQL >= 8" BLOCK;
            CheckRule "$XSS >= 8" BLOCK;
            CheckRule "$RFI >= 8" BLOCK;
            CheckRule "$TRAVERSAL >= 4" BLOCK;
            CheckRule "$EVADE >= 4" BLOCK;

            proxy_pass http://backend_app;
        }

        location /RequestDenied {
            return 403;
        }
    }
}
```

NAXSI é mais leve que o ModSecurity+CRS, mas exige uma fase de aprendizado (`LearningMode;` + revisão dos logs) para não bloquear tráfego legítimo antes de ativar o modo de bloqueio em produção.

2.4 — Filtro rápido via `map` (último recurso, não substitui o WAF)

Útil como remendo temporário enquanto um WAF de verdade não está no ar — **não** trate isso como solução definitiva, pois regex simples é trivialmente contornável (`UNI/**/ON SELECT` , `%253C` duplo-encoded, etc.).

```

map $args $blocked_pattern {
    default 0;
    "~*(\%27)|(\\"'|)(\-\-)|(\%23)|(#)"          1; # aspas/comentários SQL
    "~*((\%3D)|=)[^\n]*((\%27)|(\\"'|)(\-\-)|(\%3B)|(;))" 1; # padrão "= ' --"
    "~*union([^\n]*)select"                      1; # UNION SELECT
    "~*<script"                                  1; # XSS óbvio
    "~*javascript:"                              1; # XSS via URI scheme
    "~*on(error|load|mouseover)\s*="             1; # handlers de evento
    inline
}

server {
    if ($blocked_pattern) {
        return 403;
    }
    ...
}

```

if dentro de location no NGINX tem comportamento inconsistente documentado ("if is evil") — usar if apenas no nível de server, como acima, e mantê-lo simples (uma única condição de retorno), evita a maioria dos problemas.

2.5 — Rate limiting e limites de tamanho

Não impede um único payload malicioso, mas dificulta scanners automatizados (sqlmap, XSSStrike) que dependem de centenas/milhares de requisições por segundo para testar variações de payload.

```

http {
    limit_req_zone $binary_remote_addr zone=perip:10m rate=10r/s;
    client_max_body_size 1m; # reduz superfície de payloads grandes em POST

    server {
        location / {
            limit_req zone=perip burst=20 nodelay;
            ...
        }
    }
}

```


2.6 — Testando a configuração

```
# Recarregar após qualquer mudança
sudo nginx -t && sudo systemctl reload nginx

# Teste de XSS refletido
curl -s "http://localhost/search?q=<script>alert(1)</script>" -o /dev/null -w "%{http_code}\n"
# esperado com WAF ativo: 403

# Teste de SQL Injection clássico
curl -s "http://localhost/product?id=1%27%20R%20%271%27%3D%271" -o /dev/null -w "%{http_code}\n"
# esperado com WAF ativo: 403

# Confirmar que tráfego legítimo continua passando (checagem de falso positivo)
curl -s "http://localhost/search?q=notebook" -o /dev/null -w "%{http_code}\n"
# esperado: 200
```

Para uma varredura mais completa que um punhado de `curl`, use OWASP ZAP ou `sqlmap --batch --level=3` apontando para um ambiente de teste (nunca produção sem autorização).

2.7 — Bloco de hardening legado (estilo WordPress), anotado linha a linha

O bloco abaixo é um padrão muito difundido de "hardening" de `nginx.conf` / `vhost`, comumente encontrado em guias de proteção de WordPress e réplicas do antigo firewall "4G/5G" adaptadas para NGINX. Ele **não usa ModSecurity** — é inteiramente baseado em `add_header`, `location` com regex e variáveis `if`. Vale como camada adicional (principalmente contra scanners automatizados e exploits antigos e conhecidos), mas tem limitações importantes explicadas ao final de cada bloco. Trate-o como **complemento** ao CRS da seção 2.2, não como substituto.

a) Headers de segurança (estilo "clássico")

```
add_header X-Frame-Options SAMEORIGIN;
add_header X-Content-Type-Options nosniff;
add_header X-XSS-Protection "1; mode=block";
```

- `X-Frame-Options SAMEORIGIN` — permite que a página seja carregada em `<iframe>` **apenas por páginas do mesmo domínio**. É mais permissivo que o `DENY` usado na seção 2.1 — use `SAMEORIGIN` quando o próprio site precisa se enquadrar (embutir uma página sua dentro de outra página sua, ex.: preview interno), e `DENY` quando isso nunca deveria acontecer.
- `X-Content-Type-Options nosniff` — igual ao explicado na seção 2.1.

- `X-XSS-Protection "1; mode=block"` — ativa o filtro de XSS refletido **legado** dos navegadores antigos (Chrome/Safari pré-2019, IE). **Hoje esse header é ineficaz**: o Chrome e o Edge removeram esse filtro do motor de renderização (ele próprio virou vetor de ataque em alguns casos — o chamado "XSS Auditor bypass"), e navegadores modernos simplesmente ignoram o valor. Mantenha-o apenas por compatibilidade com clientes/scanners de compliance antigos que ainda o exigem — a proteção real contra XSS vem da **Content-Security-Policy**, não deste header. Se não houver exigência de compliance legada, prefira `add_header X-XSS-Protection "0";` (desativado explicitamente) combinado com uma CSP forte.

b) CSP/HSTS comentados — duas variantes com propósitos opostos

```
#add_header Referrer-Policy          "strict-origin-when-cross-origin" always;
#add_header Content-Security-Policy  "default-src 'self'; script-src 'self'; object-src
'none'" always;
#add_header Permissions-Policy       "geolocation=(), microphone=(), camera=()" always;
#add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

Este primeiro bloco comentado é a variante **restritiva**: `default-src 'self'` bloqueia qualquer recurso (script, imagem, fonte, etc.) que não venha do próprio domínio. É a que efetivamente mitiga XSS, mas **quebra** qualquer site que carregue scripts de CDN, fontes do Google Fonts, pixels de analytics, etc., sem que esses domínios sejam explicitamente adicionados à política.

```
#add_header Referrer-Policy          "no-referrer-when-downgrade" always;
#add_header Content-Security-Policy  "default-src 'self' http: https: ws: wss: data: blob:
'unsafe-inline'; frame-ancestors 'self';" always;
#add_header Permissions-Policy       "interest-cohort=()" always;
#add_header Strict-Transport-Security "max-age=31536000;
#includeSubDomains" always;
```

O segundo bloco é a variante **permissiva** — e aqui está o ponto crítico: `'unsafe-inline'` dentro de `default-src` **anula praticamente toda a proteção da CSP contra XSS**, porque é exatamente a execução de `<script>` inline injetado que a CSP existe para bloquear. Essa variante só é útil como "modo de transição" (site legado com muito JS/CSS inline que ainda não foi migrado) — nunca como configuração final. Além disso, a segunda linha do `Strict-Transport-Security` está **quebrada**: o valor foi dividido em duas linhas sem continuação (`\` ao final da primeira), o que em NGINX faria a segunda linha (`#includeSubDomains" always;`) ser interpretada como uma diretiva solta e gerar erro de sintaxe ao descomentar — junte tudo em uma única linha:

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

`Strict-Transport-Security` (**HSTS**) não tem relação direta com XSS/SQLi, mas está no grupo de headers de segurança "padrão" — ele força o navegador a **sempre** usar HTTPS para o domínio pelos

próximos `max-age` segundos, mesmo que o usuário digite `http://`. Ative apenas depois de confirmar que o HTTPS está 100% funcional no domínio e em todos os subdomínios (`includeSubDomains`), pois o navegador passará a recusar `http://` mesmo se o certificado quebrar depois.

Recomendação: descomente uma única variante (a restritiva, ajustada aos domínios reais de CDN/fontes/analytics que o site usa), nunca as duas, e nunca a variante com `'unsafe-inline'` em produção definitiva.

c) Bloqueio de dotfiles (exceto ACME/well-known)

```
location ~ /\.(!well-known) {
    deny all;
}
```

O regex usa uma *negative lookahead* (`(?!well-known)`): bloqueia qualquer caminho que comece com um ponto (`.git`, `.env`, `.htaccess`, `.svn`) **exceto** `/.well-known/`, que precisa ficar acessível porque é o caminho padrão usado pela validação ACME do Let's Encrypt e por outros mecanismos de descoberta (ex.: `/.well-known/security.txt`). Sem essa exceção, a renovação automática de certificado TLS quebraria.

d) Locations do módulo PageSpeed

```
location ~ "\.pagespeed\.([a-z]\.)?[a-z]{2}\.[^.]{10}\.[^.]+\" {
    add_header "" "";
}
location ~ "^/pagespeed_static/" { }
location ~ "^/ngx_pagespeed_beacon$" { }
```

Não são regras de segurança — são exigidas pelo módulo `ngx_pagespeed` (otimização automática de HTML/CSS/JS da Google) para não interferir nas URLs internas que ele gera. Só inclua isso se o `ngx_pagespeed` estiver de fato instalado e ativo; caso contrário, são `location` mortos e podem ser removidos.

e) Bloquear execução de PHP em diretórios de upload (WordPress)

```
location ~* /wp-includes/.*\.(php|php3|php4|php5|php6|phps|phtml)$ { deny all; }
location ~* /wp-content/.*\.(php|php3|php4|php5|php6|phps|phtml)$ { deny all; }
location ~* /modules/.*\.(php|php3|php4|php5|php6|phps|phtml)$ { deny all; }
location ~* /skins/.*\.(php|php3|php4|php5|php6|phps|phtml)$ { deny all; }
```

Este é um dos controles **mais importantes** do bloco inteiro contra execução remota de código (RCE), o passo seguinte comum depois de uma SQLi ou upload malicioso mal validado: se um atacante consegue colocar um arquivo `.php` dentro de `wp-content/uploads/` (via um plugin de upload vulnerável, por

exemplo), essas regras impedem que o **PHP-FPM execute esse arquivo** mesmo que ele esteja fisicamente no disco — a requisição é barrada antes de chegar ao interpretador PHP. `wp-includes` e `wp-content` nunca deveriam precisar executar PHP arbitrário fora dos arquivos legítimos do próprio WordPress/tema/plugin.

f) Cache de estáticos + IP allowlist do XML-RPC

```
location ~* \.(jpg|jpeg|gif|png|css|js|ico|xml)$ {
    access_log      off;
    log_not_found   off;
    expires         30d;
    gzip_static     on;
}

location ~* /xmlrpc.php$ {
    allow 192.0.64.0/18;
    deny all;
}
```

O primeiro bloco é puramente de performance (cache de 30 dias para estáticos, sem poluir o log de acesso). `gzip_static on` exige que existam versões `.gz` pré-comprimidas dos arquivos ao lado dos originais (módulo `ngx_http_gzip_static_module`).

O segundo bloco restringe `/xmlrpc.php` — histórico ponto de abuso do WordPress para ataques de **pingback DDoS amplificado** e força bruta de login em massa — a um único range de IP.

Atenção — erro comum: `192.0.64.0/18` é um range de exemplo. Se você copiar essa regra sem trocar pelo(s) IP(s) reais da sua equipe/CI/monitoramento, **você mesmo ficará bloqueado** do XML-RPC (o que pode ser exatamente o que você quer, se ninguém legítimo precisa dele — nesse caso prefira simplesmente `deny all;` sem `allow`, que é mais claro sobre a intenção).

g) Bloquear métodos HTTP incomuns

```
if ($request_method !~ ^(GET|POST|HEAD|PUT|DELETE|OPTIONS)$ ) {
    return 444;
}
```

Rejeita qualquer verbo HTTP fora da lista (`TRACE` , `CONNECT` , métodos inventados por scanners). O código **444** é específico do NGINX: fecha a conexão TCP imediatamente, sem enviar nenhuma resposta — mais hostil a scanners automatizados do que um `403` normal, que ao menos confirma que o servidor está vivo e respondendo.

h) Mais bloqueios de execução PHP e de metadados sensíveis

```
location ~* /(?:uploads|fotos|upload|files|wp-content|wp-includes|akismet)/.*.php$ {
    deny all; access_log off; log_not_found off;
}
location ~ /\.(svn|git)/ * {
    deny all; access_log off; log_not_found off;
}
location ~ /\.ht {
    deny all; access_log off; log_not_found off;
}
location ~ /\.user.ini {
    deny all; access_log off; log_not_found off;
}
```

Reforça o item (e) para outros diretórios de upload comuns (uploads , fotos , files), e bloqueia:

- **.svn / .git** — expor o diretório de controle de versão permite reconstruir todo o histórico do código-fonte (incluindo credenciais commitadas por engano no passado).
- **.ht*** (.htaccess , .htpasswd) — arquivos de configuração do Apache que não deveriam nunca ser servidos como conteúdo estático.
- **.user.ini** — arquivo de configuração PHP por diretório (equivalente ao .htaccess para PHP-FPM); se um atacante consegue gravar um .user.ini num diretório de upload, pode alterar comportamento do PHP (ex.: auto_prepend_file) para conseguir execução de código.

i) Rate limiting e Basic Auth no admin (comentados = opcionais)

```
#location ~ (wp-login|xmlrpc)\.php {
#    limit_req zone=one burst=2 nodelay;
#    include fastcgi_params;
#    fastcgi_pass 127.0.0.1:9000;
#    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
#    limit_req_status 444;
#}

#location /wp-admin {
#    auth_basic "Administrator's Area";
#    auth_basic_user_file /etc/nginx/.htpasswd;
#}
```

Duas camadas extras opcionais, comentadas por padrão porque exigem configuração prévia (a zona `limit_req_zone` precisa existir em `http {}`, e o `.htpasswd` precisa ser gerado com `htpasswd`):

- **Rate limiting em `wp-login / xmlrpc`** — limita tentativas de força bruta de login.
- **HTTP Basic Auth em `/wp-admin`** — adiciona uma segunda camada de senha *antes mesmo* de chegar à tela de login do WordPress. Eficaz, mas lembre-se de excluir `admin-ajax.php` da regra se algum plugin do front-end depender dele (senão o site público quebra).

j) Bloqueio por User-Agent (scanners e ferramentas de automação)

```
if ($http_user_agent ~* (nmap|fimap|netsparker|nikto|wikto|sf|sqlmap|bsqlbf|w3af|acunetix|
havij|appscan)) {
    return 403;
}
if ($http_user_agent ~* (Wget) ) { return 403; }
if ($http_user_agent ~* LWP::Simple|BBBike|wget) { return 403; }
if ($http_user_agent ~* msnbot|scrapbot) { return 403; }
```

Bloqueia pela string de User-Agent de scanners de segurança conhecidos (`nmap` , `sqlmap` , `acunetix` , `nikto` , `w3af` , `havij`) e algumas bibliotecas de automação (`LWP::Simple` , `Wget`). **Eficácia limitada:** User-Agent é um header enviado pelo próprio cliente — qualquer atacante minimamente competente troca o User-Agent para um valor de navegador comum (`curl -A "Mozilla/5.0..."`) e contorna isso instantaneamente. Trate como um filtro para os scans mais "preguiçosos"/automatizados, não como controle de segurança confiável. Note também que o próprio arquivo comenta a exclusão de `wget` da lista principal com a observação "*não desative wget se você precisa dele para cron jobs*" — um lembrete de que regras de bloqueio por assinatura têm efeitos colaterais em automações legítimas do próprio servidor.

k) Bloqueio de SQL Injection e ataques comuns via query string

```
set $block_sql 0;
if ($query_string ~* "(union.*select|select.*from|insert.*into|drop.*table|or.*1=1)") {
    set $block_sql 1;
}
if ($block_sql = 1) { return 403; }
```

O padrão "`set $var 0 → if marca $var 1 → if ($var = 1) return 403`" é usado porque o NGINX **não suporta `if / else if` encadeado nem múltiplas condições num único `if`** — essa é a forma idiomática de simular "se qualquer uma destas condições bater, bloqueie". Este bloco especificamente pega assinaturas clássicas de SQLi (`UNION SELECT` , `SELECT ... FROM` , `INSERT INTO` , `DROP TABLE` , `OR 1=1`) na query string.

```

if ($query_string ~* "union.*select.*\(") { set $block_attack 1; }
if ($query_string ~* "select.*from") { set $block_attack 1; }
if ($query_string ~* "concat.*\(") { set $block_attack 1; }
if ($query_string ~* "<script>|%3Cscript%3E") { set $block_attack 1; }
if ($request_uri ~* "(base64_|localhost|127\.0\.0\.1|file:///|input|data:text/html)") { set
$block_attack 1; }
if ($block_attack = 1) { return 403; }

```

Amplia a cobertura: `CONCAT(` (usado para exfiltrar múltiplas colunas numa SQLi via `UNION SELECT CONCAT(...)`), `<script>` cru ou URL-encoded (XSS), e um conjunto de padrões associados a **SSRF/LFI** — `file://` (leitura de arquivo local via wrapper de protocolo), `data:text/html` (payload de dados embutido), `localhost / 127.0.0.1` (tentativa de acessar serviços internos via um proxy/fetch vulnerável na aplicação, o mesmo tipo de teste da seção **LM-07** do manual de Lateral Movement, se você tiver esse documento à mão).

Os dois blocos seguintes (`$block_sql_injections` e `$block_file_injections`) são, na prática, **redundantes** com os anteriores — sobreposição típica de configs que foram coladas de fontes diferentes ao longo do tempo. `$block_file_injections` merece destaque à parte:

```

set $block_file_injections 0;
if ($query_string ~ "[a-zA-Z0-9]=http://") { set $block_file_injections 1; }
if ($query_string ~ "[a-zA-Z0-9]=(\\.\\.//?)+") { set $block_file_injections 1; }
if ($query_string ~ "[a-zA-Z0-9]=/(\\[a-z0-9_\\.//?)+") { set $block_file_injections 1; }
if ($block_file_injections = 1) { return 403; }

```

Isso mira **Remote/Local File Inclusion (RFI/LFI)** de aplicações PHP antigas que faziam `include($_GET['page'])` sem validação: `?page=http://site-do-atacante.com/shell.txt` (RFI) ou `?page=../../../../etc/passwd` (LFI via path traversal). É um resquício da era do PHP `register_globals / allow_url_include`, mas ainda vale como camada extra se alguma aplicação legada rodar atrás desse NGINX.

I) Exploits específicos e spam via query string

```

if ($query_string ~ "GLOBALS(=|\\[|\\%[0-9A-Z]{0,2})") { set $block_common_exploits 1; }
if ($query_string ~ "_REQUEST(=|\\[|\\%[0-9A-Z]{0,2})") { set $block_common_exploits 1; }
if ($query_string ~ "proc/self/enviro") { set $block_common_exploits 1; }
if ($query_string ~ "mosConfig_[a-zA-Z]{1,21}(=|\\%3D)") { set $block_common_exploits 1; }
if ($query_string ~ "base64_(en|de)code\\(.*)") { set $block_common_exploits 1; }

```

Cada padrão aqui é a assinatura de uma vulnerabilidade **específica e histórica**: `GLOBALS[ / _REQUEST[` exploram a injeção das superglobais do PHP em apps que confiavam demais no `register_globals` (removido do PHP desde a versão 5.4, mas o padrão sobrevive em configs copiadas); `proc/self/enviro`

é uma técnica de LFI-para-RCE que lê o próprio ambiente do processo Apache/PHP (que inclui o `User-Agent` da requisição) para injetar código PHP via header e depois incluí-lo; `mosConfig_` é específico do **Joomla** (injeção no arquivo de configuração); `base64_(en|de)code()` tenta pegar payloads ofuscados em base64 — mas isso é facilmente evadido com qualquer outro encoding (hex, ROT13, dupla camada) e pode gerar falsos positivos legítimos (aplicações que usam base64 em parâmetros por design).

O bloco de `$block_spam` (palavras como `viagra`, `cialis`, `tramadol`) não é uma defesa contra XSS/SQLi — é um filtro anti-spam de SEO, usado contra bots que tentam injetar links de spam farmacêutico em campos de busca/comentários indexáveis. Mantém-se aqui por tradição histórica do gist original, mas é opcional e pode ser removido sem impacto de segurança.

m) Locations com regex genéricas de "caractere perigoso" — a parte mais arriscada do bloco

```
location ~* "(eval\()\" { deny all; }
location ~ "(\\|\\.\\.\\.|\\.\\.\\.|/|~|`|<|>|\\|)\" { deny all; }
location ~* "(\\'|\\")(\\.\\.\\.)(drop|insert|md5|select|union)\" { deny all; }
location ~* "(https?|ftp|php):/" { deny all; }
location ~* "(=\\|'|=\\%27|/\\|'/?|\\)\\.\" { deny all; }
location ~ "(\\{\\}\\}|\\(\\/\\(|\\.\\.\\.|\\+\\+\\+|\\|\\|\"\\|\\|)\" { deny all; }
location ~ "(~|`|<|>|:|;|%|\\|\\s|\\{|\\}|\\[\\]|\\|)\" { deny all; }
```

Essas linhas bloqueiam pela **presença do padrão em qualquer parte da URI** (não só na query string). A penúltima e a última merecem atenção redobrada:

- `(https?|ftp|php):/` bloqueia qualquer URI contendo um protocolo embutido — mitigação genérica contra RFI/SSRF via wrapper de protocolo, mas também vai bloquear, por exemplo, um parâmetro legítimo `?redirect=https://parceiro.com` se por acaso ele trafegar sem URL-encoding correto no path.
- **A última linha é a mais perigosa de todo o arquivo:** `(~|`|<|>|:|;|%|\\|\\s|\\{|\\}|\\[\\]|\\|)` bloqueia **qualquer URI que contenha um dos caracteres**: til, crase, <, >, dois-pontos, ponto-e-vírgula, % (!), barra invertida, espaço, chaves ou colchetes. Como % é o próprio caractere de **URL-encoding**, e dois-pontos aparece em URLs com porta ou em alguns formatos de data/hora em query strings, essa regra tem alto potencial de **falso positivo** contra tráfego legítimo (qualquer parâmetro url-encoded contém %XX`). Teste exaustivamente em modo de log antes de ativar essa linha específica em produção — muitos administradores a comentam ou removem por esse motivo.


```
location ~* "(&pbs=0|_vti_|\\(null\\)|\\{\\$itemURL\\}|echo(.*)kae|boot\\.ini|etc/passwd|eval\\(|
self/envIRON|(wp-)?config\\.|cgi-|muieblack)" { deny all; }
location ~* "/(^$|mobiquo|phpinfo|shell|sqlpatch|thumb|thumb_editor|thumbopen|timthumb|
webshell|config|configuration)\\.php" { deny all; }
location ~* "\\.(aspx?|bash|bak?|cfg|cgi|dll|exe|git|hg|ini|jsp|log|mdb|out|sql|svn|swp|tar|
rdf)$" { deny all; }
```

Os últimos três blocos são listas de assinaturas nomeadas:

- `_vti_` é um resquício do FrontPage Server Extensions (Microsoft, décadas atrás); `boot.ini` / `etc/passwd` são alvos clássicos de LFI/path traversal para provar leitura arbitrária de arquivo; `muieblack` é a assinatura de um bot de scraping/exploit chinês bastante documentado em blocklists antigas; `(wp-)?config\\.` tenta impedir acesso direto a `wp-config.php` / `config.php` (que contêm credenciais de banco).
- `timthumb`, `thumb`, `thumb_editor`, `mobiquo`, `webshell` — nomes de scripts PHP de plugins/temas com **vulnerabilidades de RCE amplamente exploradas** no ecossistema WordPress/Joomla entre 2011-2015 (o caso do TimThumb foi um dos maiores incidentes em massa da época). Bloquear pelo nome do arquivo é uma mitigação **reativa** — só funciona para vulnerabilidades já catalogadas.
- A última linha bloqueia **extensões de arquivo** que nunca deveriam ser servidas publicamente: backups (`.bak` , `.tar`), controle de versão (`.git` , `.svn` , `.hg`), configuração (`.cfg` , `.ini`), banco de dados (`.mdb` , `.sql`), logs (`.log`), executáveis (`.exe` , `.dll`), e scripts de outras stacks que não deveriam existir no seu servidor (`.aspx` , `.jsp` , `.cgi`) — se aparecerem numa requisição, é sinal de scanner de reconhecimento tentando descobrir se algum desses arquivos foi esquecido no diretório web.

Resumo de risco deste bloco (seção 2.7): é uma coleção real e historicamente eficaz contra ataques automatizados e oportunistas, mas (1) é 100% blocklist — o problema estrutural descrito na seção 1.3 se aplica integralmente; (2) tem pelo menos uma regra (o character-class genérico do item m) com risco real de falso positivo; (3) mistura dezenas de `if` no nível de `server`, o que é aceitável em termos de correção mas tem custo de performance por requisição maior que um único `SecRule` do ModSecurity avaliando as mesmas assinaturas de forma otimizada. Use como **camada extra de baixo custo de implantação**, sempre com um período de observação em log antes de ir para bloqueio total, e mantenha o CRS (seção 2.2) como a defesa primária.

3. Apache HTTP Server

3.1 — Headers de segurança com `mod_headers`

```
# /etc/apache2/conf-available/security-headers.conf
<IfModule mod_headers.c>
    Header always set X-Content-Type-Options "nosniff"
    Header always set X-Frame-Options "DENY"
    Header always set Referrer-Policy "strict-origin-when-cross-origin"
    Header always set Permissions-Policy "geolocation=(), microphone=(), camera=()"
    Header always set Content-Security-Policy "default-src 'self'; script-src 'self'; style-
src 'self' 'unsafe-inline'; object-src 'none'; base-uri 'self'; frame-ancestors 'none';"
    Header always unset X-XSS-Protection
</IfModule>
```

```
sudo a2enmod headers
sudo a2enconf security-headers
sudo systemctl reload apache2
```

O significado de cada diretiva é o mesmo descrito na seção 2.1 — a única diferença é a sintaxe (`Header always set` em vez de `add_header ... always`).

3.2 — ModSecurity 2 + OWASP CRS

Instalação (Debian/Ubuntu):

```
sudo apt install libapache2-mod-security2
sudo a2enmod security2
sudo cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf
```

Edite `/etc/modsecurity/modsecurity.conf` e troque o modo de detecção para bloqueio:

```
SecRuleEngine On
SecRequestBodyAccess On
SecResponseBodyAccess Off
```

Comece com `SecRuleEngine DetectionOnly` em produção por alguns dias, revise `/var/log/apache2/modsec_audit.log` em busca de falsos positivos, e só então troque para `On`. Ativar bloqueio direto sem essa fase é a causa nº 1 de "o WAF quebrou o site" em produção.

Baixar e habilitar o CRS:

```
cd /etc/modsecurity
sudo git clone https://github.com/coreruleset/coreruleset.git crs
sudo cp crs/crs-setup.conf.example crs/crs-setup.conf
```

Inclua no `/etc/apache2/mods-enabled/security2.conf`:

```
<IfModule security2_module>
    SecDataDir /var/cache/modsecurity
    IncludeOptional /etc/modsecurity/*.conf
    IncludeOptional /etc/modsecurity/crs/crs-setup.conf
    IncludeOptional /etc/modsecurity/crs/rules/*.conf
</IfModule>
```

```
sudo systemctl restart apache2
```

Regras próprias de exemplo (mesma lógica da seção 2.2 — a sintaxe do ModSecurity é idêntica em NGINX e Apache, pois é o mesmo motor):

```
<IfModule security2_module>
    SecRule ARGS "@detectXSS" \
        "id:2001,phase:2,deny,status:403,log,msg:'XSS Attack Detected'"

    SecRule ARGS "@detectSQLi" \
        "id:2002,phase:2,deny,status:403,log,msg:'SQL Injection Attack Detected'"
</IfModule>
```

3.3 — Filtro rápido via `mod_rewrite` (último recurso)

```
# vhost ou .htaccess
RewriteEngine On

# Bloqueia tags <script> na query string (encoded ou não)
RewriteCond %{QUERY_STRING} (<|%3C)\s*script [NC,OR]

# Bloqueia handlers de evento inline (onerror=, onload=, etc.)
RewriteCond %{QUERY_STRING} on(error|load|mouseover)\s*= [NC,OR]

# Bloqueia padrões clássicos de UNION-based SQLi
RewriteCond %{QUERY_STRING} union([\n]*)select [NC,OR]

# Bloqueia comentários SQL usados para evadir filtros (-- , #, /*)
RewriteCond %{QUERY_STRING} (\-\-|\%23|#|/\*) [NC]

RewriteRule ^(.*)$ - [F,L]
```

Mesma ressalva da seção 2.4: isto é um curativo, não uma solução — mantenha o ModSecurity+CRS como a defesa primária e trate essas regras apenas como bloqueio de assinaturas óbvias enquanto o WAF completo não está disponível.

3.4 — Testando a configuração

```
sudo apachectl configtest && sudo systemctl reload apache2

curl -s "http://localhost/search?q=<script>alert(1)</script>" -o /dev/null -w "%{http_code}\n"
# esperado com WAF/regras ativas: 403

curl -s "http://localhost/product?id=1%27%20R%20%271%27%3D%271" -o /dev/null -w "%{http_code}\n"
# esperado: 403

curl -s "http://localhost/search?q=notebook" -o /dev/null -w "%{http_code}\n"
# esperado: 200 (garante que não há falso positivo bloqueando busca legítima)
```

4. O que a camada de servidor web NÃO resolve

Nenhuma configuração de NGINX/Apache corrige a causa raiz. Trate as seções 2 e 3 como **mitigação de risco enquanto o código é corrigido**, nunca como substituto de:

VULNERABILIDADE	CORREÇÃO REAL (NA APLICAÇÃO)
SQL Injection	Prepared statements / queries parametrizadas (nunca concatenar input em SQL) — em qualquer linguagem/framework (PDO com bind params em PHP, <code>?</code> placeholders em JDBC, ORMs configurados corretamente).
XSS Refletido/ Armazenado	Output encoding contextual (HTML entity encoding ao inserir em HTML, JS string escaping ao inserir em <code><script></code> , URL encoding em atributos <code>href</code>) — depende de onde o dado é impresso, não existe um único "escape" universal.
XSS DOM-based	Evitar <code>innerHTML</code> / <code>document.write</code> com dados não confiáveis; usar <code>textContent</code> ou sanitização explícita (ex.: DOMPurify) no client-side.
Ambos	Validação de input do tipo allowlist (formato esperado, tamanho, charset) na entrada, além do escaping na saída.

Um WAF bem configurado (ModSecurity + CRS) reduz drasticamente a exploração automatizada e dá tempo para corrigir o código com segurança — mas payloads o suficientemente ofuscados eventualmente contornam qualquer regra baseada em assinatura. É por isso que a OWASP classifica controles de servidor/WAF como **camada complementar**, nunca como controle primário, no OWASP Top 10 (A03 — Injection e A03 anterior — XSS).

5. Cheat-sheet de diretivas

OBJETIVO	NGINX	APACHE
Header CSP	<code>add_header Content-Security-Policy "...";</code>	<code>Header always set Content-Security-Policy "..."</code>
Bloquear MIME sniffing	<code>add_header X-Content-Type-Options "nosniff" always;</code>	<code>Header always set X-Content-Type-Options "nosniff"</code>
Bloquear framing (clickjacking)	<code>add_header X-Frame-Options "DENY" always;</code>	<code>Header always set X-Frame-Options "DENY"</code>
Ativar WAF (ModSecurity)	<code>modsecurity on;</code> <code>modsecurity_rules_file ...;</code>	<code>SecRuleEngine On</code> (dentro de <code>security2.conf</code>)
Modo somente-log (calibragem)	<code>SecRuleEngine DetectionOnly</code> (mesmo arquivo <code>modsec</code>)	<code>SecRuleEngine DetectionOnly</code>
Deteção estrutural de XSS	<code>SecRule ARGS "@detectXSS" ...</code>	<code>SecRule ARGS "@detectXSS" ...</code>

OBJETIVO	NGINX	APACHE
Detecção estrutural de SQLi	<code>SecRule ARGS "@detectSQLi" ...</code>	<code>SecRule ARGS "@detectSQLi" ...</code>
Regex rápida na query string	<code>map \$args \$x { "~*padrão" 1; } / if (\$x) { return 403; }</code>	<code>RewriteCond %{QUERY_STRING} padrão [NC] + RewriteRule .* - [F]</code>
Rate limiting	<code>limit_req_zone + limit_req</code>	<code>mod_ratelimit ou mod_evasive</code>
Limitar tamanho do corpo	<code>client_max_body_size 1m;</code>	<code>LimitRequestBody 1048576</code>
Recarregar config	<code>nginx -t && systemctl reload nginx</code>	<code>apachectl configtest && systemctl reload apache2</code>

6. Referências

- OWASP ModSecurity Core Rule Set (CRS) — <https://coreruleset.org>
- OWASP Secure Headers Project
- OWASP Cheat Sheet Series — "Cross Site Scripting Prevention" e "SQL Injection Prevention"
- OWASP Top 10 2025 — A03: Injection
- Documentação oficial: nginx.org/en/docs , httpd.apache.org/docs
- NAXSI — <https://github.com/nbs-system/naxsi>
- ModSecurity-nginx connector — <https://github.com/owasp-modsecurity/ModSecurity-nginx>